

Project Report

Database Management Systems

(Course code: 25TU03MJM0)



Submitted by:

Sai Sri Charan Chunduri

ID: 25WU0102235

Specialization: Artificial Intelligence and Machine Learning

Section: Whales

(B. Tech 2025-29)

School of Technology

Submitted on: 07/10/2026

Index

S. No.	Contents	Page No.
1	Abstract	1
2	Introduction	1
3	Problem Statement	2
4	Objectives	2
5	Scope	4
6	Software Requirements	4
7	Hardware	9
8	ER Diagram	10
9	Schema Relationships	12
10	Normalization	12
11	Data Dictionary	13
12	SQL Commands	17
13	Queries with Commands	17
14	Query Given During Presentation-II	22
15	UI Design and Screenshots	23
16	Implementation Details	28
17	Database Connectivity	29
18	Key Code Snippet – Data Retrieval	29
19	Key Code Snippet – Insert Operation	29
20	Database Triggers	30
21	Testing	30
22	Conclusion and Future Enhancements	31
23	References	32
24	Appendix: GitHub Repository Link	32

1.Abstract

The Fuel Station Sales and Tank Inventory Management System is an application created to manage the fuel station affairs. The project aims to design and develop a database that will contain the necessary information about sales, inventory, suppliers, attendants, customers, tank stocks, delivery, and invoices.

MySQL was used to design and develop the database, while Python and FastAPI were employed to develop an API dashboard. The database has primary and foreign keys, constraints, and triggers to ensure referential integrity and control the activities of the database users. The database employs triggers to automate the management process of the fuel station, including the management of tank inventory, customer credits, and invoice payments.

In addition, the application will be used to monitor fuel inventory, sales, and tank stock, track payments owed and received, and view suppliers, customers, and other details on a dashboard. The database that powers the web application can be described as a combination of databases management and dashboard; the project offers solutions for managing a fuel station, which is beneficial because it speeds up the management processes and reduces the chances of errors in the processes.

2.Introduction

- A database-driven application that streamlines fuel station operations through efficient organizational processes.
- An integrated system that manages fuel data, tank inventory, sales, suppliers, deliveries, customers, and payments.
- The application uses MySQL database to ensure data is organized in a structured manner.
- It also uses Python programming language and FastAPI web framework that make it easy to use.
- The application further uses database constraints and triggers to guarantee data quality and accuracy.
- Finally, the application expedites the management process, minimizes manual handling of data and reduces transaction errors.
- Supports **real-time monitoring and centralized access** to important fuel station information.

3.Problem statement

Traditional or manual operation of fuel stations may encounter challenges in recording fuel stock, sale, delivery, suppliers, customers, invoice, and payments information. The records kept separately incur mistakes in calculation, storage, and acquisition of fuel, leading to disorder and mismanagement.

In addition, the lack of a reliable database management system to store and update data automatically causes irregularities in stock management, inability to record credit sales, track payments, and prepare reports based on the gathered data.

Therefore, a database system that fulfills the following specifications is required:

- Store integrated data concerning fuel station's operations.
- Monitor the fuel stock in the tank.
- Record sales and delivery of the fuel.
- Manage suppliers, attendants, and customers.
- Monitor invoices and payments.
- Maintain up-to-date inventory and payments records through triggers.
- Enforce referential integrity via constraints.
- Allow web access to the database through a user-friendly interface.
- Facilitate report generation for effective decision-making.

The project proposes to design a system that employs MySQL database and FastAPI web application to fulfill these specifications and provide an organized, reliable, and integrated platform for fuel station operators to monitor sales and tank inventory.

4.Objectives

The Fuel Station Sales and Tank Inventory Management System objectives are:

1. Centralized database
2. Fuel inventory management
3. Sales record keeping
4. Supplier and delivery management
5. Customer and credit management
6. Data integrity and validation
7. Tank stock management
8. Invoice and payment tracking
9. Web application interface

10. Report generation and decision support

Each objective will be accomplished by:

1. Centralized Data Management

To create a centralized database that will help store and manage all the required information in a structured manner.

2. Fuel Inventory Management

To keep track of all fuel types, storage tanks, tank sizes, and inventory stock data.

3. Sales Transaction Management

To record all sales transaction data including the fuel type sold, the amount sold at each price, the payment mode, the dispenser number, and shift.

4. Supplier and Delivery Management

To store and manage suppliers' data and record all the purchases made from approved suppliers. This includes the amount purchased at a time, cost of the fuel purchase, and invoice number for each delivery.

5. Customer and Credit Management

To record and store the data of all credit customers including their credit limit, credit sales, invoices, and amounts owed.

6. Data Integrity and Validation

To develop a database that is reliable and correctly stores the system information using primary and foreign keys, constraints, and triggers to maintain the entity's integrity.

7. Tank Stock Management

To design the system in a way that it can track and update the stock in each tank after a sale or upon receiving a new delivery of fuel.

8. Invoice and Payment Tracking

To record and track all the invoices issued to customers and manage the payments outstanding, partially paid, and cleared invoices.

9. Web application interface

To develop a user-friendly web application interface using FastAPI, HTML, CSS, and JavaScript to view and manage the system's database.

10. Report generation and decision support

To develop reports that include stock, sales, payments, suppliers, and customers' data to support the management in concluding the business's progress.

5.Scope

The Fuel Station Sales and Tank Inventory Management System scope includes the following:

Fuel inventory management: This covers managing different fuel types, storage tanks, tank capacity, and current stock levels.

Sales management: This involves recording and managing sales data including the quantity of fuel sold, prices, dispensers, shifts, and payment methods.

Supplier management: This entails maintaining supplier information and tracking fuel deliveries at the gas station.

Attendant and shift management: This covers managing attendants' duties, including dispensers' meter readings, and shifts.

Customer management: This involves maintaining customer information, including customers on credit.

Invoice and payment management: This entails generating fuel invoices and managing payments, including collecting outstanding amounts.

Automated database management: This includes using triggers to update inventory and payment information and enforcing rules in the database.

Reporting: The system will generate reports on inventory, sales, payments, suppliers, and customers.

Web-based interface: This will provide a portal for managing the fuel station.

The system can be further expanded to include analytics and alerts features, accessed via mobile apps, and have additional management functionalities.

6.Software Requirements

1. MySQL

MySQL is used as the primary Relational Database Management System (RDBMS) for the project.

Our project uses MySQL to:

- Store all fuel station information in relational tables.
- Manage the 12 main entities/tables including fuel_type, tank, supplier, delivery, sale, attendant, shift, invoice, and payment.
- Establish relationships between tables using Primary Keys and Foreign Keys.
- Enforce data integrity using constraints such as NOT NULL, UNIQUE, CHECK, and FOREIGN KEY.
- Automatically perform important operations using database triggers.
- Manage fuel stock when deliveries and sales take place.
- Maintain invoice and payment information.
- Role: Core database and data management system.

2. Python

Python is used as the main programming language for the application's backend. In our case, it is implemented by a number of Python scripts.

They are responsible for:

- Implementing the app's backend logic;
- Processing the requests from the web interface;
- Communicating with the MySQL database;
- Retrieving and processing data from the database;
- Sending the database records to the web interface;
- Performing any operations (adding, deleting, viewing, updating the records).

Role: Backend programming and application logic.

3. FastAPI

FastAPI is the Python web framework used to develop the backend of our web application.

In our project, FastAPI:

- creates the web application and its routes
- connects the front-end interface to the MySQL database
- handles HTTP requests from the various pages.
- provides routes for modules including:
 1. dashboard
 2. inventory
 3. sales
 4. suppliers
 5. customers
 6. deliveries
 7. invoices
 8. payments

9. reports

- processes form submissions such as adding or deleting records
- uses Jinja2 templates to display database information dynamically.

Role: Backend web framework and communication layer.

4. HTML

HTML (HyperText Markup Language) is a standard language used in the creation of the web interface. In our project, HTML is applied to:

construct dashboard pages,

- menus,
- tables,
- forms,
- buttons,
- cards,
- statistics,
- inventory,
- sales and payments,

etc. For instance, the dashboard is built using HTML codes to display particular data, such as total sales, revenue, fuel stock, suppliers, customers, and money at the moment.

Role: HTML is to structure the web interface.

5. CSS

Cascading Style Sheets (CSS) define the presentation and styling of the web application.

In our project, CSS is used to:

- Implement the dark navy theme
- Add electric-blue accents
- Introduce fuel-orange highlights
- Style glassmorphism cards
- Design tables and forms
- Style buttons and status badges
- Create sidebar navigation menus
- Ensure responsive design
- Implement hover effects and animations
- Establish consistent spacing, typography, and colors

It creates the modern and professional interface of a fuel-station management application.

Role: Styling and design.

6. JavaScript

JavaScript is used to add interactive and dynamic functionality to the web interface.

In our project, JavaScript can be used for:

- Interactive UI elements.
- Dynamic page behavior.
- Search and filtering functionality.
- Updating interface elements without changing the overall page structure.
- Creating interactive visual components and dashboard features.
- Improving the overall user experience.

Role: Client-side interaction and dynamic functionality.

7. Jinja2

Jinja2 is a template engine used with FastAPI to dynamically generate HTML pages.

Instead of writing database information manually into HTML, Jinja2 allows the application to insert information retrieved from MySQL.

This allows the dashboard and other pages to automatically display **current database information**.

Role: Dynamic connection between backend data and HTML pages.

8. MySQL Connector/Python MySQL Connector/Python is used to connect the Python application and the MySQL database.

As for the features we have used in our project, they are:

- Establishing a connection to MySQL.
- Performing SQL queries from Python.
- Obtaining database records.
- Inserting new records.
- Deleting records.
- Processing database transactions.
- Closing the database connection, after the operations are done.

The basic connection is realized through Python code like:

```
mysql.connector.connect(
    host="localhost",
    user="root",
    password="...",
    database="fuel_station_management"
)
```

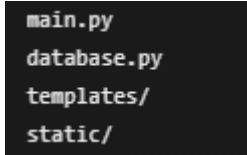
Role: Communication between Python/FastAPI and MySQL.

9. Visual Studio Code Essay

The Visual Studio Code (VS Code) is the tool used to develop the project. It is used for the following purposes:

- To write and edit Python code
- To develop FastAPI routes
- To design and develop HTML templates
- To write CSS and JavaScript
- To manage project files
- To debug
- To run and test FastAPI application

The project consists of the following files:



```
main.py
database.py
templates/
static/
```

Role: Code development and project management.

10. Web Browser

A modern **web browser** is used to access and operate the application.

The browser is where one performs various tasks such as:

- Viewing dashboards
- Checking fuel inventory
- Managing suppliers and customers
- Viewing sales and deliveries
- Managing invoices and payments
- Reports
- The system also has CRUD operations under the interface

Purpose: It is the role whereby a user accesses the application through a browser or website.

7.Hardware

Component	Minimum Requirement	Recommended
Processor	Intel Core i3 / equivalent	Intel Core i5 or above
RAM	4 GB	8 GB or above
Storage	10 GB free space	20 GB or above
Display	1366 × 768	Full HD (1920 × 1080)
Keyboard	Standard keyboard	Standard keyboard
Mouse	Standard mouse	Standard mouse
Network	Local network/internet	Stable internet connection



8.ER Diagram

The **Entity–Relationship (ER) Diagram** represents the structure of the Fuel Station Sales and Tank Inventory Management System. It shows the major entities, their attributes, and the relationships between them.

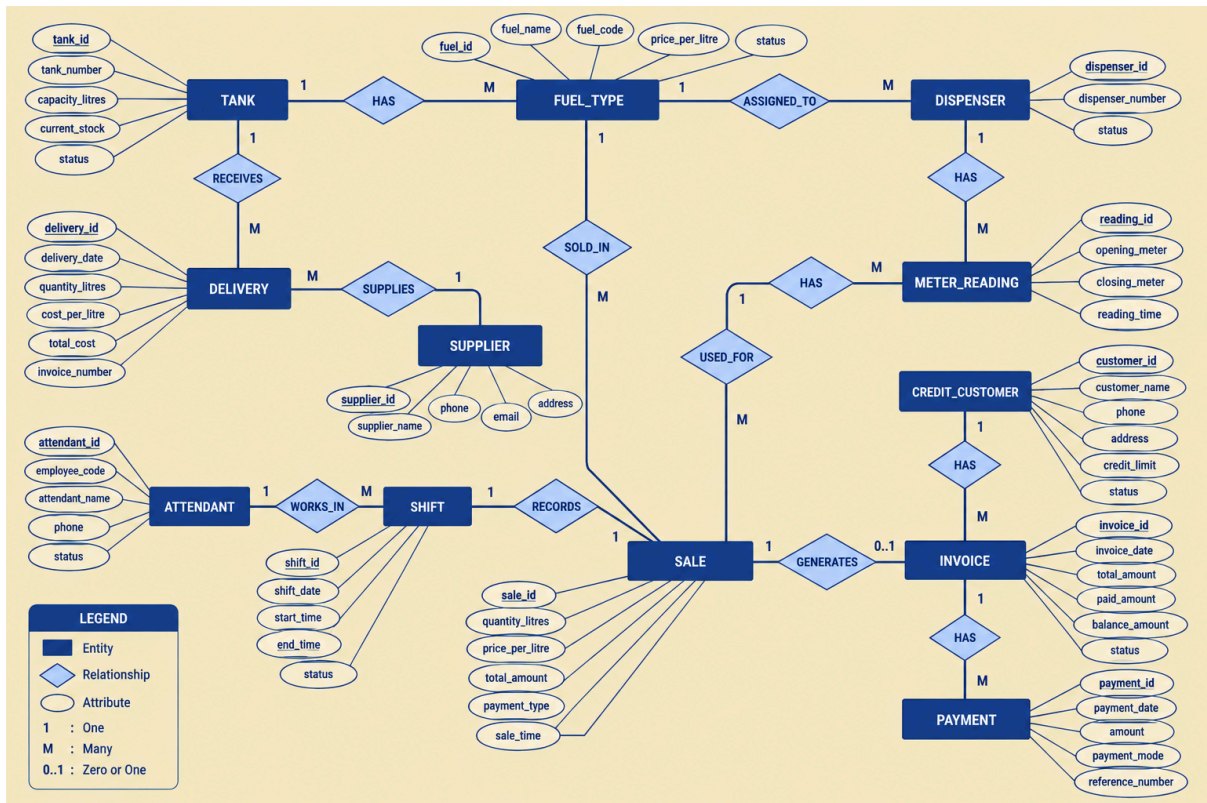
Main Entities

The system consists of the following **12 principal entities**:

1. **FUEL_TYPE** – Stores different types of fuel and their prices.
2. **TANK** – Maintains storage tank capacity and current fuel stock.
3. **SUPPLIER** – Stores supplier details.
4. **DELIVERY** – Records fuel deliveries received from suppliers.
5. **DISPENSER** – Maintains fuel dispensing unit information.
6. **ATTENDANT** – Stores fuel station employee details.
7. **SHIFT** – Records attendant shifts and their timings.
8. **METER_READING** – Stores opening and closing dispenser meter readings.
9. **SALE** – Records fuel sales transactions.
10. **CREDIT_CUSTOMER** – Stores customers who purchase fuel on credit.
11. **INVOICE** – Maintains invoices generated for credit sales.
12. **PAYMENT** – Records payments made against invoices.

Major Relationships

Relationship	Cardinality
FUEL_TYPE → TANK	1 : N
FUEL_TYPE → DISPENSER	1 : N
SUPPLIER → DELIVERY	1 : N
TANK → DELIVERY	1 : N
ATTENDANT → SHIFT	1 : N
DISPENSER → METER_READING	1 : N
SHIFT → METER_READING	1 : N
SHIFT → SALE	1 : N
DISPENSER → SALE	1 : N
FUEL_TYPE → SALE	1 : N
CREDIT_CUSTOMER → INVOICE	1 : N
SALE → INVOICE	1 : 1
INVOICE → PAYMENT	1 : N



Primary Key and Foreign Key Mapping

Table	Primary Key	Foreign Key(s)
FUEL_TYPE	fuel_id	—
TANK	tank_id	fuel_id → FUEL_TYPE
SUPPLIER	supplier_id	—
DELIVERY	delivery_id	supplier_id → SUPPLIER, tank_id → TANK
DISPENSER	dispenser_id	fuel_id → FUEL_TYPE
ATTENDANT	attendant_id	—
SHIFT	shift_id	attendant_id → ATTENDANT
METER_READING	reading_id	dispenser_id → DISPENSER, shift_id → SHIFT
SALE	sale_id	shift_id → SHIFT, dispenser_id → DISPENSER, fuel_id → FUEL_TYPE
CREDIT_CUSTOMER	customer_id	—
INVOICE	invoice_id	customer_id → CREDIT_CUSTOMER, sale_id → SALE
PAYMENT	payment_id	invoice_id → INVOICE

9.Schema Relationships

The foreign keys establish relationships between the tables and maintain **referential integrity**. For example:

- `TANK.fuel_id` references `FUEL_TYPE.fuel_id`.
- `DELIVERY.supplier_id` references `SUPPLIER.supplier_id`.
- `DELIVERY.tank_id` references `TANK.tank_id`.
- `SHIFT.attendant_id` references `ATTENDANT.attendant_id`.
- `SALE.fuel_id` references `FUEL_TYPE.fuel_id`.
- `INVOICE.customer_id` references `CREDIT_CUSTOMER.customer_id`.
- `PAYMENT.invoice_id` references `INVOICE.invoice_id`.

This relational structure ensures that the database is **organized, connected, and consistent**, while reducing unnecessary data duplication.

10.Normalization

Our database is designed up to **Third Normal Form (3NF)**. The normalization is applied across the following tables:

- **1NF:** Applied to **all 12 tables**, as every attribute contains atomic values and each record has a unique primary key.
- **2NF:** Applied to **all 12 tables**, since the non-key attributes depend on their respective primary keys. The tables mainly use single-attribute primary keys, so there are no partial dependencies.
- **3NF:** Applied by separating related information into different tables. For example, **fuel information** is stored in `FUEL_TYPE`, **supplier information** in `SUPPLIER`, **tank information** in `TANK`, **attendant information** in `ATTENDANT`, **customer information** in `CREDIT_CUSTOMER`, **invoice information** in `INVOICE`, and **payment information** in `PAYMENT`. These tables are connected using foreign keys.

Therefore, the complete database is normalized up to 3NF, with the 3NF structure helping reduce redundancy and maintain data consistency.

11.Data Dictionary

The **Data Dictionary** provides a detailed description of the tables, attributes, data types, and constraints used in the **Fuel Station Sales and Tank Inventory Management System**. The database consists of **12 main tables**.

FUEL_TYPE

Attribute	Data Type	Constraints	Description
<code>fuel_id</code>	INT	PK, AUTO_INCREMENT	Unique identifier for fuel
<code>fuel_name</code>	VARCHAR(50)	NOT NULL, UNIQUE	Name of the fuel
<code>fuel_code</code>	VARCHAR(10)	NOT NULL, UNIQUE	Unique fuel code
<code>price_per_litre</code>	DECIMAL(10,2)	NOT NULL, CHECK > 0	Price per litre
<code>status</code>	VARCHAR(10)	NOT NULL, DEFAULT 'ACTIVE'	Fuel status

TANK

Attribute	Data Type	Constraints	Description
<code>tank_id</code>	INT	PK, AUTO_INCREMENT	Unique tank identifier
<code>tank_number</code>	VARCHAR(20)	NOT NULL, UNIQUE	Tank identification number
<code>fuel_id</code>	INT	FK, NOT NULL	Fuel stored in the tank
<code>capacity_litres</code>	DECIMAL(10,2)	NOT NULL, CHECK > 0	Maximum tank capacity
<code>current_stock</code>	DECIMAL(10,2)	NOT NULL, DEFAULT 0	Current fuel stock
<code>status</code>	VARCHAR(10)	NOT NULL, DEFAULT 'ACTIVE'	Tank status

SUPPLIER

Attribute	Data Type	Constraints	Description
<code>supplier_id</code>	INT	PK, AUTO_INCREMENT	Unique supplier identifier
<code>supplier_name</code>	VARCHAR(100)	NOT NULL	Supplier name
<code>phone</code>	VARCHAR(15)	NOT NULL, UNIQUE	Supplier contact number
<code>email</code>	VARCHAR(100)	UNIQUE	Supplier email
<code>address</code>	VARCHAR(255)	NOT NULL	Supplier address

DELIVERY

Attribute	Data Type	Constraints	Description
<code>delivery_id</code>	INT	PK, AUTO_INCREMENT	Unique delivery identifier
<code>supplier_id</code>	INT	FK, NOT NULL	Supplier providing the fuel
<code>tank_id</code>	INT	FK, NOT NULL	Tank receiving the fuel
<code>delivery_date</code>	DATE	NOT NULL	Date of delivery
<code>quantity_litres</code>	DECIMAL(10,2)	NOT NULL, CHECK > 0	Quantity delivered
<code>cost_per_litre</code>	DECIMAL(10,2)	NOT NULL, CHECK > 0	Cost per litre
<code>total_cost</code>	DECIMAL(12,2)	NOT NULL, CHECK > 0	Total delivery cost
<code>invoice_number</code>	VARCHAR(50)	NOT NULL, UNIQUE	Delivery invoice number

DISPENSER

Attribute	Data Type	Constraints	Description	
<code>dispenser_id</code>	INT	PK, AUTO_INCREMENT	Unique dispenser identifier	
<code>dispenser_number</code>	VARCHAR(20)	NOT NULL, UNIQUE	Dispenser number	
<code>fuel_id</code>	INT	FK, NOT NULL	Fuel supplied by dispenser	
<code>status</code>	VARCHAR(10)	NOT NULL, DEFAULT 'ACTIVE'	Dispenser status	

ATTENDANT

Attribute	Data Type	Constraints	Description
<code>attendant_id</code>	INT	PK, AUTO_INCREMENT	Unique attendant identifier
<code>employee_code</code>	VARCHAR(20)	NOT NULL, UNIQUE	Employee identification code
<code>attendant_name</code>	VARCHAR(100)	NOT NULL	Attendant name
<code>phone</code>	VARCHAR(15)	NOT NULL, UNIQUE	Contact number
<code>status</code>	VARCHAR(10)	NOT NULL, DEFAULT 'ACTIVE'	Employment status

SHIFT

Attribute	Data Type	Constraints	Description
shift_id	INT	PK, AUTO_INCREMENT	Unique shift identifier
attendant_id	INT	FK, NOT NULL	Attendant assigned to shift
shift_date	DATE	NOT NULL	Date of shift
start_time	TIME	NOT NULL	Shift starting time
end_time	TIME	NULL	Shift ending time
status	VARCHAR(10)	NOT NULL, DEFAULT 'OPEN'	Shift status

METER_READING

Attribute	Data Type	Constraints	Description
reading_id	INT	PK, AUTO_INCREMENT	Unique reading identifier
dispenser_id	INT	FK, NOT NULL	Related dispenser
shift_id	INT	FK, NOT NULL	Related shift
opening_meter	DECIMAL(12,2)	NOT NULL, CHECK ≥ 0	Opening meter reading
closing_meter	DECIMAL(12,2)	NOT NULL, CHECK ≥ 0	Closing meter reading
reading_time	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Reading timestamp

SALE

Attribute	Data Type	Constraints	Description
sale_id	INT	PK, AUTO_INCREMENT	Unique sale identifier
shift_id	INT	FK, NOT NULL	Shift during which sale occurred
dispenser_id	INT	FK, NOT NULL	Dispenser used
fuel_id	INT	FK, NOT NULL	Fuel sold
quantity_litres	DECIMAL(10,2)	NOT NULL, CHECK > 0	Quantity sold
price_per_litre	DECIMAL(10,2)	NOT NULL, CHECK > 0	Selling price
total_amount	DECIMAL(12,2)	NOT NULL, CHECK > 0	Total sale amount
payment_type	VARCHAR(10)	NOT NULL	Payment method
sale_time	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Sale timestamp

CREDIT_CUSTOMER

Attribute	Data Type	Constraints	Description
customer_id	INT	PK, AUTO_INCREMENT	Unique customer identifier
customer_name	VARCHAR(100)	NOT NULL	Customer name
phone	VARCHAR(15)	NOT NULL, UNIQUE	Customer contact number
address	VARCHAR(255)	NOT NULL	Customer address
credit_limit	DECIMAL(12,2)	NOT NULL, CHECK ≥ 0	Maximum credit limit
status	VARCHAR(10)	NOT NULL, DEFAULT 'ACTIVE'	Customer status

INVOICE

Attribute	Data Type	Constraints	Description
invoice_id	INT	PK, AUTO_INCREMENT	Unique invoice identifier
customer_id	INT	FK, NOT NULL	Credit customer
sale_id	INT	FK, UNIQUE, NOT NULL	Related sale
invoice_date	DATE	NOT NULL	Invoice date
total_amount	DECIMAL(12,2)	NOT NULL, CHECK > 0	Total invoice amount
paid_amount	DECIMAL(12,2)	NOT NULL, DEFAULT 0	Amount paid
balance_amount	DECIMAL(12,2)	NOT NULL, CHECK ≥ 0	Outstanding amount
status	VARCHAR(10)	NOT NULL, DEFAULT 'PENDING'	Invoice payment status

PAYMENT

Attribute	Data Type	Constraints	Description
payment_id	INT	PK, AUTO_INCREMENT	Unique payment identifier
invoice_id	INT	FK, NOT NULL	Related invoice
payment_date	DATE	NOT NULL	Date of payment
amount	DECIMAL(12,2)	NOT NULL, CHECK > 0	Amount paid
payment_mode	VARCHAR(10)	NOT NULL	Payment method
reference_number	VARCHAR(50)	UNIQUE	Payment reference

12.SQL Commands

Command	Category	Purpose
CREATE DATABASE	DDL	Creates the database
USE	DDL	Selects the database
CREATE TABLE	DDL	Creates database tables
ALTER TABLE	DDL	Modifies table structure
INSERT	DML	Adds records
SELECT	DML	Retrieves records
UPDATE	DML	Modifies records
DELETE	DML	Removes records
JOIN	DML/Query	Retrieves related data from multiple tables

13.Queries with commands

Query 1: Fuel-wise Total Sales

Question: Find the total quantity of fuel sold and total revenue generated for each fuel type.

```
SELECT
    f.fuel_name,
    SUM(s.quantity_litres) AS total_litres_sold,
    SUM(s.total_amount) AS total_revenue
FROM sale s
JOIN fuel_type f
    ON s.fuel_id = f.fuel_id
GROUP BY f.fuel_id, f.fuel_name;
```

fuel_name	total_litres_sold	total_revenue
CNG	311.00	27368.00
Diesel	1307.00	121289.60
Electric Charging	248.00	4588.00
Petrol	1313.00	138521.50
Premium Diesel	471.00	46393.50
Premium Petrol	550.00	63250.00

6 rows in set (0.019 sec)

Query 2: Current Stock of Each Fuel

Question: Display the current stock along with the fuel name and tank capacity.

```

SELECT
    f.fuel_name,
    t.tank_number,
    t.capacity_litres,
    t.current_stock,
    ROUND((t.current_stock / t.capacity_litres) * 100, 2) AS stock_percentage
FROM tank t
JOIN fuel_type f
    ON t.fuel_id = f.fuel_id
ORDER BY stock_percentage DESC;

```

fuel_name	tank_number	capacity_litres	current_stock	stock_percentage
Diesel	TANK-02	25000.00	22693.00	90.77
Premium Petrol	TANK-03	15000.00	12450.00	83.00
Petrol	TANK-01	20000.00	16187.00	80.94
CNG	TANK-04	20000.00	8189.00	40.95
Electric Charging	TANK-05	15000.00	5952.00	39.68
Premium Diesel	TANK-06	18000.00	7029.00	39.05

6 rows in set (0.085 sec)

Query 3: Sales Above Average Sale Amount

Question: Find sales whose total amount is greater than the average sale amount.

```

SELECT
    sale_id,
    fuel_id,
    quantity_litres,
    total_amount,
    payment_type
FROM sale

```

```

WHERE total_amount > (
    SELECT AVG(total_amount)
    FROM sale
)
ORDER BY total_amount DESC;

```

sale_id	fuel_id	quantity_litres	total_amount	payment_type
3	2	80.00	7424.00	CARD
300	6	61.00	6008.50	CARD
304	6	58.00	5713.00	UPI
4	2	60.00	5568.00	CASH
297	6	55.00	5417.50	CARD
1	1	50.00	5275.00	CASH
302	6	52.00	5122.00	CASH
299	6	48.00	4728.00	CASH
303	6	45.00	4432.50	CARD
2	1	40.00	4220.00	UPI
296	6	42.00	4137.00	CREDIT
305	6	40.00	3940.00	CASH
282	4	44.00	3872.00	UPI
298	6	37.00	3644.50	UPI
278	4	41.00	3608.00	CARD
5	3	30.00	3450.00	CREDIT
284	4	38.00	3344.00	CARD
301	6	33.00	3250.50	CREDIT
280	4	35.00	3080.00	CASH
276	4	32.00	2816.00	CREDIT
281	4	29.00	2552.00	CREDIT
285	4	27.00	2376.00	UPI
277	4	25.00	2200.00	CASH
37	1	19.00	2004.50	CARD
283	4	22.00	1936.00	CASH
86	1	18.00	1899.00	CARD
11	1	18.00	1899.00	CARD
102	1	18.00	1899.00	CARD
136	1	18.00	1899.00	CASH
130	3	16.00	1840.00	CASH
117	1	17.00	1793.50	CARD
32	1	17.00	1793.50	UPI
147	1	17.00	1793.50	CASH
61	1	17.00	1793.50	CARD
115	3	15.00	1725.00	CASH
121	1	16.00	1688.00	CARD
41	1	16.00	1688.00	CASH
17	1	16.00	1688.00	UPI
151	1	16.00	1688.00	CASH
82	1	16.00	1688.00	CARD
107	1	16.00	1688.00	UPI
74	2	18.00	1670.40	CREDIT
48	2	18.00	1670.40	CASH

80	3	14.00	1610.00	CASH
279	4	18.00	1584.00	UPI
26	1	15.00	1582.50	CARD
132	1	15.00	1582.50	UPI
76	1	15.00	1582.50	CASH
97	1	15.00	1582.50	CARD
52	1	15.00	1582.50	UPI
68	2	17.00	1577.60	CARD
19	2	17.00	1577.60	CREDIT
109	2	17.00	1577.60	CREDIT
265	3	13.00	1495.00	CREDIT
100	3	13.00	1495.00	UPI
145	3	13.00	1495.00	UPI
50	3	13.00	1495.00	CARD
139	2	16.00	1484.80	CREDIT
28	2	16.00	1484.80	UPI
93	2	16.00	1484.80	CARD
54	2	16.00	1484.80	CREDIT
22	1	14.00	1477.00	CARD
257	1	14.00	1477.00	CREDIT
111	1	14.00	1477.00	CASH
72	1	14.00	1477.00	UPI
142	1	14.00	1477.00	CARD
231	1	14.00	1477.00	CREDIT
192	1	14.00	1477.00	CREDIT
46	1	14.00	1477.00	CARD
64	2	15.00	1392.00	CREDIT
124	2	15.00	1392.00	CREDIT
43	2	15.00	1392.00	UPI
143	2	15.00	1392.00	CASH
9	2	15.00	1392.00	CASH
88	2	15.00	1392.00	UPI
155	3	12.00	1380.00	CASH
35	3	12.00	1380.00	CREDIT
70	3	12.00	1380.00	CASH
250	3	12.00	1380.00	CREDIT
190	3	12.00	1380.00	CREDIT
246	1	13.00	1371.50	CREDIT
172	1	13.00	1371.50	CREDIT
122	1	13.00	1371.50	CASH
87	1	13.00	1371.50	CASH
137	1	13.00	1371.50	UPI
16	1	13.00	1371.50	CASH
66	1	13.00	1371.50	CASH
222	1	13.00	1371.50	CREDIT
212	1	13.00	1371.50	CREDIT
272	1	13.00	1371.50	CREDIT
237	1	13.00	1371.50	CREDIT
196	1	13.00	1371.50	CREDIT
57	1	13.00	1371.50	CARD

93 rows in set (0.031 sec)

Query 4: Supplier-wise Delivery Summary

Question: Find the total quantity of fuel delivered by each supplier.

```

SELECT
    s.supplier_name,
    COUNT(d.delivery_id) AS total_deliveries,
    SUM(d.quantity_litres) AS total_quantity_delivered,
    SUM(d.total_cost) AS total_delivery_cost
FROM supplier s
JOIN delivery d
    ON s.supplier_id = d.supplier_id
GROUP BY s.supplier_id, s.supplier_name
ORDER BY total_quantity_delivered DESC;

```

supplier_name	total_deliveries	total_quantity_delivered	total_delivery_cost
Bharat Petroleum	1	6000.00	522000.00
Indian Oil Corporation	1	5000.00	492500.00
Hindustan Petroleum	1	4000.00	432000.00

3 rows in set (0.012 sec)

Query 5: Payment Method Analysis

Question: Find the number of sales and total revenue generated through each payment method.

```
SELECT
    payment_type,
    COUNT(*) AS number_of_sales,
    SUM(total_amount) AS total_revenue
FROM sale
GROUP BY payment_type
ORDER BY total_revenue DESC;
```

payment_type	number_of_sales	total_revenue
CREDIT	157	167068.90
CASH	53	87102.80
CARD	50	82825.50
UPI	45	64413.40

4 rows in set (0.009 sec)

Query 6: Outstanding Invoices

Question: Display customers who have outstanding invoice balances.

```
SELECT
    c.customer_name,
    i.invoice_id,
    i.total_amount,
    i.paid_amount,
    i.balance_amount,
    i.status
FROM credit_customer c
JOIN invoice i
    ON c.customer_id = i.customer_id
WHERE i.balance_amount > 0
ORDER BY i.balance_amount DESC;
```

customer_name	invoice_id	total_amount	paid_amount	balance_amount	status
XYZ Logistics	516	4137.00	0.00	4137.00	PENDING
Sri Sai Travels	517	3250.50	0.00	3250.50	PENDING
ABC Transport	1	3450.00	1500.00	1950.00	PARTIAL
ABC Transport	393	1670.40	0.00	1670.40	PENDING
ABC Transport	386	1577.60	0.00	1577.60	PENDING
ABC Transport	404	1577.60	0.00	1577.60	PENDING
ABC Transport	380	1484.80	0.00	1484.80	PENDING
ABC Transport	397	1484.80	0.00	1484.80	PENDING
ABC Transport	383	1392.00	0.00	1392.00	PENDING
ABC Transport	395	1392.00	0.00	1392.00	PENDING
ABC Transport	401	1380.00	0.00	1380.00	PENDING
ABC Transport	360	1371.50	0.00	1371.50	PENDING
ABC Transport	378	1299.20	0.00	1299.20	PENDING
ABC Transport	387	1299.20	0.00	1299.20	PENDING
ABC Transport	389	1299.20	0.00	1299.20	PENDING
ABC Transport	366	1266.00	0.00	1266.00	PENDING
ABC Transport	368	1206.40	0.00	1206.40	PENDING
ABC Transport	402	1206.40	0.00	1206.40	PENDING
ABC Transport	371	1160.50	0.00	1160.50	PENDING
ABC Transport	357	1150.00	0.00	1150.00	PENDING
ABC Transport	373	1113.60	0.00	1113.60	PENDING
ABC Transport	384	1113.60	0.00	1113.60	PENDING
ABC Transport	391	1113.60	0.00	1113.60	PENDING
ABC Transport	400	1113.60	0.00	1113.60	PENDING
ABC Transport	375	1055.00	0.00	1055.00	PENDING
ABC Transport	359	1020.80	0.00	1020.80	PENDING
ABC Transport	379	1020.80	0.00	1020.80	PENDING
ABC Transport	405	1020.80	0.00	1020.80	PENDING
ABC Transport	340	1477.00	492.33	984.67	PARTIAL
ABC Transport	370	949.50	0.00	949.50	PENDING
ABC Transport	364	928.00	0.00	928.00	PENDING
ABC Transport	382	928.00	0.00	928.00	PENDING
ABC Transport	392	928.00	0.00	928.00	PENDING
ABC Transport	399	928.00	0.00	928.00	PENDING
ABC Transport	342	1300.00	460.00	920.00	PARTIAL
ABC Transport	362	920.00	0.00	920.00	PENDING
ABC Transport	403	920.00	0.00	920.00	PENDING
ABC Transport	310	1371.50	457.17	914.33	PARTIAL
ABC Transport	320	1371.50	457.17	914.33	PARTIAL
ABC Transport	336	1371.50	457.17	914.33	PARTIAL
ABC Transport	353	1299.20	433.07	866.13	PARTIAL
ABC Transport	331	1266.00	422.00	844.00	PARTIAL
ABC Transport	355	1266.00	422.00	844.00	PARTIAL
ABC Transport	376	844.00	0.00	844.00	PENDING
ABC Transport	312	1265.00	421.67	843.33	PARTIAL
ABC Transport	347	1265.00	421.67	843.33	PARTIAL
ABC Transport	363	835.20	0.00	835.20	PENDING
ABC Transport	381	835.20	0.00	835.20	PENDING
ABC Transport	394	835.20	0.00	835.20	PENDING
ABC Transport	367	805.00	0.00	805.00	PENDING
ABC Transport	406	805.00	0.00	805.00	PENDING
ABC Transport	329	1206.40	402.13	804.27	PARTIAL
ABC Transport	325	1160.50	386.83	773.67	PARTIAL
ABC Transport	345	1160.50	386.83	773.67	PARTIAL
ABC Transport	322	1150.00	383.33	766.67	PARTIAL

Query 7: Attendant-wise Sales

Question: Find the total sales handled by each attendant.

SELECT

a.attendant_name,

COUNT(s.sale_id) AS total_sales,

SUM(s.total_amount) AS total_revenue

FROM attendant a

JOIN shift sh

ON a.attendant_id = sh.attendant_id

JOIN sale s

ON sh.shift_id = s.shift_id

GROUP BY a.attendant_id, a.attendant_name

ORDER BY total_revenue DESC;

attendant_name	total_sales	total_revenue
Kiran Kumar	5	25937.00
Vishal Reddy	10	17972.00
Rohan Kumar	10	16094.00
Suman Kumar	10	15898.50
Rahul Reddy	10	15642.60
Naresh Rao	10	15477.00
Abhay Reddy	10	14433.50
Madhav Kumar	10	14235.00
Kunal Kumar	10	13715.00
Madan Rao	10	13421.00
Ganesh Kumar	10	13366.40
Kishore Rao	10	13358.40
Vivek Rao	10	12970.10
Vijay Kumar	10	12888.00
Uday Kumar	10	12676.50
Sanjay Kumar	10	12553.30
Naveen Kumar	10	12539.00
Surya Reddy	10	12363.50
Prasad Rao	10	12100.00
Manish Kumar	10	11830.50
Surendra Reddy	10	11595.00
Shiva Rao	10	11001.00
Akhil Reddy	10	10888.50
Raghu Reddy	10	10540.50
Rajesh Reddy	10	10336.00
Lokesh Reddy	10	10288.00
Abhinav Rao	10	10002.70
Harish Rao	10	9936.00
Sathish Rao	10	9703.50
Krishna Kumar	10	8925.80
Mohan Rao	10	8722.30

31 rows in set (0.034 sec)

Query 8: Fuel Types with Stock Below 50%

Question: Identify tanks where the current stock is below 50% of their capacity.

```
SELECT
    f.fuel_name,
    t.tank_number,
    t.current_stock,
    t.capacity_litres
FROM tank t
JOIN fuel_type f
    ON t.fuel_id = f.fuel_id
WHERE t.current_stock < (t.capacity_litres * 0.50);
```

fuel_name	tank_number	current_stock	capacity_litres
CNG	TANK-04	8189.00	20000.00
Electric Charging	TANK-05	5952.00	15000.00
Premium Diesel	TANK-06	7029.00	18000.00

3 rows in set (0.008 sec)

Query 9: Top 5 Highest-Value Sales

Question: Find the five highest-value sales.

```
SELECT
    s.sale_id,
    f.fuel_name,
    s.quantity_litres,
    s.price_per_litre,
    s.total_amount,
    s.payment_type
FROM sale s
JOIN fuel_type f
    ON s.fuel_id = f.fuel_id
ORDER BY s.total_amount DESC
LIMIT 5;
```

sale_id	fuel_name	quantity_litres	price_per_litre	total_amount	payment_type
3	Diesel	80.00	92.80	7424.00	CARD
300	Premium Diesel	61.00	98.50	6008.50	CARD
304	Premium Diesel	58.00	98.50	5713.00	UPI
4	Diesel	60.00	92.80	5568.00	CASH
297	Premium Diesel	55.00	98.50	5417.50	CARD

5 rows in set (0.010 sec)

14. Query given during Presentation-II

Retrieve the current fuel stock available in each tank and the total outstanding amount yet to be paid

```
USE fuel_station_management;
```

```
SELECT
    f.fuel_name AS Fuel_Name,
    COALESCE(t.fuel_left, 0) AS Fuel_Left_Litres,
    COALESCE(i.unpaid_amount, 0) AS Unpaid_Amount
FROM fuel_type f
```

```
LEFT JOIN (
    SELECT
        fuel_id,
        SUM(current_stock) AS fuel_left
    FROM tank
    GROUP BY fuel_id
) t ON f.fuel_id = t.fuel_id
```

```
LEFT JOIN (
    SELECT
        s.fuel_id,
        SUM(i.balance_amount) AS unpaid_amount
    FROM invoice i
    JOIN sale s ON i.sale_id = s.sale_id
    WHERE i.balance_amount > 0
    GROUP BY s.fuel_id
) i ON f.fuel_id = i.fuel_id;
```

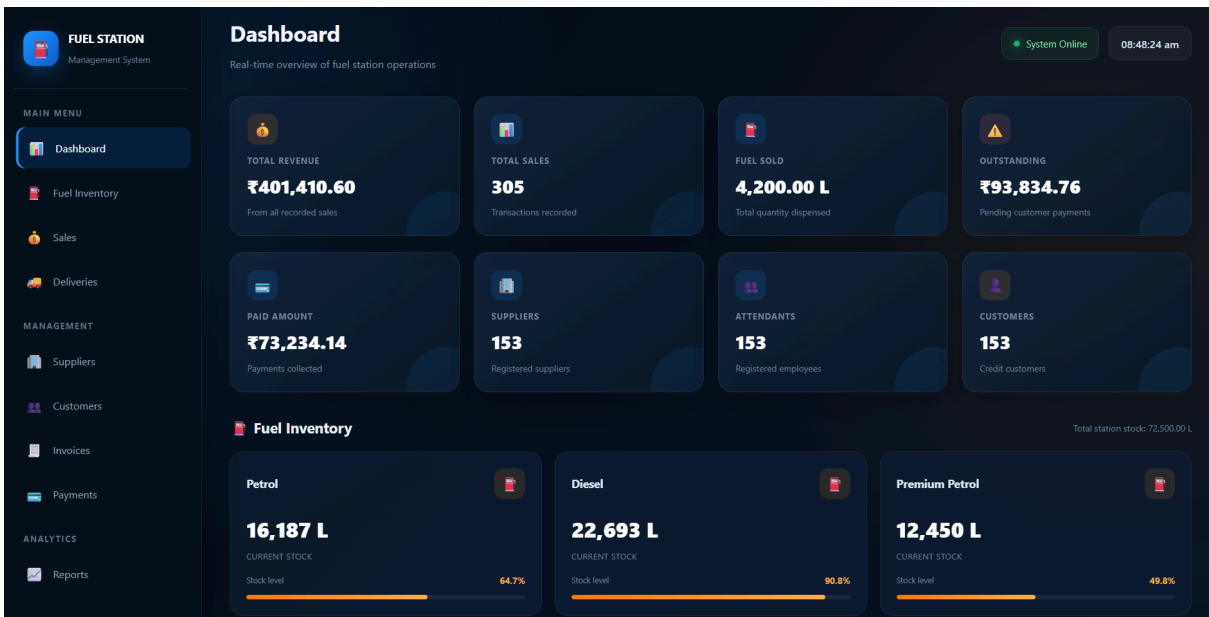
Fuel Name	Fuel Left (Litres)	Unpaid Amount
Petrol	17410	0.00
Diesel	23860	0.00
Premium Petrol	12970	1950.00

15.Ul design and screenshots

The Fuel Station Sales and Tank Inventory Management System provide a modern web-based user interface built using FastAPI, HTML, CSS, JavaScript, and Jinja2 template. The interface offers an intuitive and efficient way of accessing all the modules while providing essential information on the go.

Dashboard

The dashboard page contains the general information of the fuel station, including the total sales, total revenue, total fuel sold, payment due, amount paid, total suppliers, attendants, and customers. It also contains the total fuel stock, current fuel in the tank, and station modules.



Inventory

The Inventory page provides all the fuel types, fuel tanks, tank capacity, and current fuel stock.

Fuel Station Management System							
Fuel Inventory Management							
Fuel Inventory							
Live tank stock and fuel pricing							
Current Fuel Stock							
FUEL ID	FUEL NAME	FUEL CODE	TANK	CAPACITY	CURRENT STOCK	PRICE / LITRE	STATUS
1	Petrol	PET	TANK-01	20000.00 L	16187.00 L	₹105.50	ACTIVE
2	Diesel	DSL	TANK-02	25000.00 L	22693.00 L	₹92.80	ACTIVE
3	Premium Petrol	PPR	TANK-03	15000.00 L	12450.00 L	₹115.00	ACTIVE
4	CNG	CNG	TANK-04	20000.00 L	8189.00 L	₹88.00	ACTIVE
5	Electric Charging	EV	TANK-05	15000.00 L	5952.00 L	₹18.50	ACTIVE
6	Premium Diesel	PDL	TANK-06	18000.00 L	7029.00 L	₹98.50	ACTIVE

Sales

The Sales page shows all the sales data, including the fuel type, quantity, price, amount, payment type, and time of sale.

Fuel Station Management System

Sales Management

Sales Records

View all fuel sales and payment transactions

Dashboard

Recent Sales

SALE ID	FUEL	DISPENSER	QUANTITY	PRICE / L	TOTAL AMOUNT	PAYMENT TYPE	SALE TIME
305	Premium Diesel	DISP-08	40.00 L	₹98.50	₹3940.00	CASH	2026-10-06 07:47:10
304	Premium Diesel	DISP-08	58.00 L	₹98.50	₹5713.00	UPI	2026-10-06 07:47:10
303	Premium Diesel	DISP-08	45.00 L	₹98.50	₹4432.50	CARD	2026-10-06 07:47:10
302	Premium Diesel	DISP-08	52.00 L	₹98.50	₹5122.00	CASH	2026-10-06 07:47:10
301	Premium Diesel	DISP-08	33.00 L	₹98.50	₹3250.50	CREDIT	2026-10-06 07:47:10
300	Premium Diesel	DISP-08	61.00 L	₹98.50	₹6008.50	CARD	2026-10-06 07:47:10
299	Premium Diesel	DISP-08	48.00 L	₹98.50	₹4728.00	CASH	2026-10-06 07:47:10

Suppliers

The Suppliers page enables the supervisor to interact with all the suppliers, including deleting or adding new suppliers in the database.

Supplier Management

Insert, view and delete supplier records

Dashboard

+ Add New Supplier

Supplier Name

Phone Number

Email Address

Address

Add Supplier

Supplier Records

SUPPLIER ID	SUPPLIER NAME	PHONE	EMAIL	ADDRESS	ACTION
153	Indian Fuel Logistics 150	8000000150	supplier150@fuel.com	Ahmedabad	Delete
152	Indian Fuel Logistics 149	8000000149	supplier149@fuel.com	Kolkata	Delete
151	Indian Fuel Logistics 148	8000000148	supplier148@fuel.com	Delhi	Delete
150	Indian Fuel Logistics 147	8000000147	supplier147@fuel.com	Pune	Delete

Customers

This page contains all the customer's information registered in the fuel company, including the customer's credit.

Credit Customers

Insert, view and delete customer records

Dashboard

+ Add New Customer

Customer Name

Phone Number

Address

Credit Limit

Add Customer

Customer Records

CUSTOMER ID	CUSTOMER NAME	PHONE	ADDRESS	CREDIT LIMIT	STATUS	ACTION
153	ABC Transport 150	6000000150	Kolkata	₹60000.00	INACTIVE	Delete
152	ABC Transport 149	6000000149	Delhi	₹80000.00	INACTIVE	Delete
151	ABC Transport 148	6000000148	Pune	₹50000.00	INACTIVE	Delete
150	ABC Transport 147	6000000147	Mumbai	₹90000.00	INACTIVE	Delete

Deliveries

This page displays all the fuel deliveries received from the suppliers and the tanks into which they are pumped.

Fuel Station Management System

Fuel Delivery Management

Dashboard

Fuel Deliveries

Supplier deliveries and tank replenishment records

Delivery Records

DELIVERY ID	SUPPLIER	TANK	FUEL	DELIVERY DATE	QUANTITY	COST / L	TOTAL COST	INVOICE NO.
3	Hindustan Petroleum	TANK-03	Premium Petrol	2026-08-26	4000.00 L	₹108.00	₹432000.00	INV-1003
2	Bharat Petroleum	TANK-02	Diesel	2026-08-26	6000.00 L	₹87.00	₹522000.00	INV-1002
1	Indian Oil Corporation	TANK-01	Petrol	2026-08-26	5000.00 L	₹98.50	₹492500.00	INV-1001

Invoices

The Invoices page provides the invoice details, including the total invoice amount, amount paid, due amount, and clearance status.

Fuel Station Management System

Credit Invoice Management

Credit Invoices

Invoice and outstanding payment tracking

Dashboard

Invoice Records

INVOICE ID	CUSTOMER	SALE ID	INVOICE DATE	TOTAL AMOUNT	PAID AMOUNT	BALANCE	STATUS
517	Sri Sai Travels	301	2026-10-06	₹3250.50	₹0.00	₹3250.50	PENDING
516	XYZ Logistics	296	2026-10-06	₹4137.00	₹0.00	₹4137.00	PENDING
515	ABC Transport	291	2026-10-06	₹518.00	₹250.00	₹268.00	PARTIAL
514	ABC Transport 150	286	2026-10-06	₹333.00	₹150.00	₹183.00	PARTIAL
513	ABC Transport 149	281	2026-10-06	₹2552.00	₹2552.00	₹0.00	PAID
512	ABC Transport 148	276	2026-10-06	₹2816.00	₹2816.00	₹0.00	PAID
406	ABC Transport	10	2026-10-06	₹805.00	₹0.00	₹805.00	PENDING

Payments

This page stores and displays all the information on payments received and displayed in tabular format.

Payments

Record and track invoice payments

Dashboard

+ Record Payment

Select Invoice

Payment Amount

Payment Mode

Reference Number

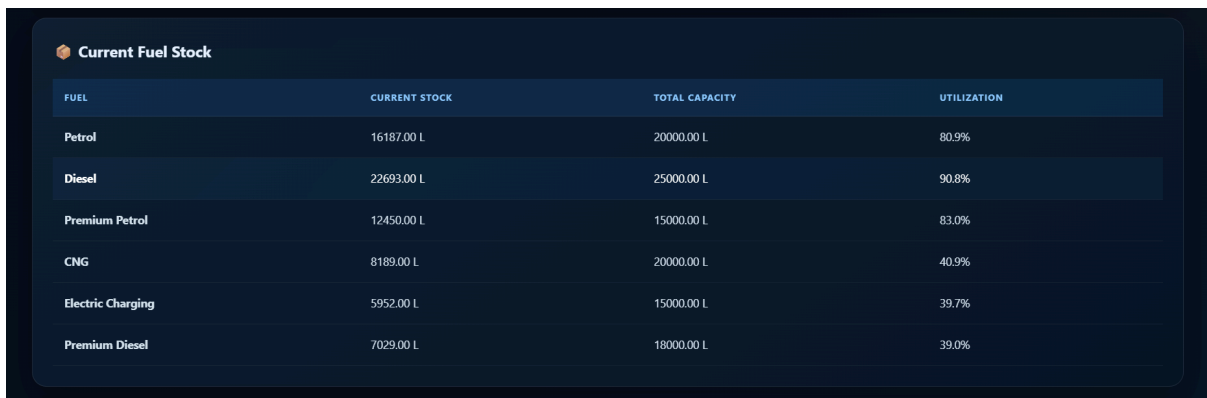
Record Payment

Payment Records

PAYMENT ID	INVOICE ID	CUSTOMER	PAYMENT DATE	AMOUNT	PAYMENT MODE	REFERENCE NUMBER
261	#515	ABC Transport	2026-10-06	₹250.00	CASH	PAY-NEW-515
260	#514	ABC Transport 150	2026-10-06	₹150.00	UPI	PAY-NEW-514
259	#513	ABC Transport 149	2026-10-06	₹2552.00	CARD	PAY-NEW-513
258	#512	ABC Transport 148	2026-10-06	₹2816.00	UPI	PAY-NEW-512

Reports

The reports page provides general information on fuel sales, inventory, and payments. It offers a simplified view of the data for easy reading and interpretation.



16.Implementation Details

Technology Stack

The system is implemented using the following technologies:

Technology	Purpose
MySQL	Database management and storage
Python	Backend programming
FastAPI	Web application framework
HTML	Web page structure
CSS	User interface styling
JavaScript	Interactive functionality

Jinja2	Dynamic data rendering
MySQL Connector/Python	Database connectivity
VS Code	Development environment

17.Database Connectivity

The application connects the FastAPI backend with the MySQL database using MySQL Connector/Python. The connection is established by specifying the database host, username, password, and database name.

Example:

```
import mysql.connector

def get_connection():
    return mysql.connector.connect(
        host="localhost",
        user="root",
        password="YOUR_MYSQL_PASSWORD",
        database="fuel_station_management"
    )
```

18.Key Code Snippet – Data Retrieval

The backend retrieves information from MySQL using SQL queries and passes the results to Jinja2 templates.

```
cursor.execute("""
    SELECT *
    FROM fuel_type
""")
```

```
fuel_stock = cursor.fetchall()
```

19.Key Code Snippet – Insert Operation

The system performs database insertion through FastAPI routes.

```

cursor.execute("""
    INSERT INTO supplier
    (supplier_name, phone, email, address)
    VALUES (%s, %s, %s, %s)
""", (supplier_name, phone, email, address))

conn.commit()

```

20.Database Triggers

Database triggers can be used to perform essential operations and ensure data integrity. Some of these operations include:

- Checking of meter readings.
- Checking of available fuel stock before a sale.
- Updating of tank stock after a sale.
- Updating of tank stock after a delivery.
- Checking of payment amount.
- Updating of invoice balances and payment status.

21.Testing

Testing was performed to verify that the database, backend, and web interface function correctly and produce the expected results.

Test ID	Test Case	Expected Result	Result
TC01	Open dashboard	Dashboard loads successfully	Pass
TC02	View fuel inventory	Current fuel stock is displayed	Pass
TC03	Add supplier	New supplier is inserted into database	Pass
TC04	Delete supplier	Selected supplier is removed	Pass
TC05	View sales	Sales records are displayed correctly	Pass
TC06	Add payment	Payment record is inserted and invoice updated	Pass

TC07	Enter invalid meter reading	System rejects invalid reading	Pass
TC08	Sell more fuel than available stock	Sale is rejected	Pass
TC09	Add fuel delivery	Tank stock is automatically increased	Pass
TC10	Make invoice payment	Paid and balance amounts are updated	Pass

The test cases were created that prove the system's ability to perform the key operations on the database and application. The application constraints and triggers ensure that the system prevents invalid transactions and maintains referential integrity.

22.Conclusion and Future Enhancements

Conclusion

- The Fuel Station Sales and Tank Inventory Management System is a centralized solution to crucial fuel station management activities. The project utilizes a MySQL relational database and a FastAPI-based web application to manage fuel inventory, sales, deliveries, suppliers, attendants, customers, invoices, and payments.
- The utilization of primary and foreign keys, constraints, and database triggers to maintain data integrity and automate vital operations while providing a web interface that offers a structured and organized way of accessing and managing the system's data.
- Thus, this project aims to reduce human effort, increase data organization, minimize transaction errors, and provide valuable insights into the management activities of a fuel station.

Future Enhancements

Further enhancements can be made to include:

- Real-time alerts for low stock tanks
- Advanced sales and revenue analyses
- Implement role-based authentication system (admin and staff)
- Automate SMS/email notifications for payments and low inventory
- Mobile app
- Online payment system
- Advanced dashboards with visual graphics
- Automated reports (daily, weekly, and monthly)
- Cloud database
- Back up and restore database

23. References

1. MySQL Documentation – MySQL Reference Manual.
2. Python Documentation – Python Software Foundation.
3. FastAPI Documentation – FastAPI Official Documentation.
4. Jinja2 Documentation – Jinja Template Documentation.
5. MDN Web Docs – HTML, CSS and JavaScript documentation.
6. W3Schools – SQL, HTML, CSS and JavaScript reference materials.

24. Appendix: GitHub Repository Link

<https://github.com/saisricharanchunduri-byte/DBMS-Course-Project>